Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	Shravan P M	
Roll Number	24110026	
Registration Number	24011101101	
Class / Section	B.Tech AI & DS - B	

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N	Input Size
F	Filter Size
P	Padding
S	Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:
- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:
- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:
- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$$

Applications:
- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{bmatrix}$$

Applications:
- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix}-1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1\end{bmatrix}$$

Applications:
- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{bmatrix}1 & 0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0 & 0\\ 0 & 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 0 & 1\end{bmatrix}$$

Applications:
- Motion simulation
- Image restoration

- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output
First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose
Input Image

$$32 \times 32 \times 3$$

Convolution Layer
- Filters = 16

- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

Loaded via `tf.keras.datasets.cifar10.load_data()`. Classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.
Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Kernel Size	Stride	Padding	Output Size
3×3	1	valid	30×30
5×5	1	valid	28×28
7×7	1	valid	26×26

Task 3

Study hyperparameters.
Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Stride	Padding	Output Size	Config #
1	valid	30×30	1
2	valid	15×15	2
1	same	32×32	3
2	same	16×16	4

Task 4
Visualize feature maps after the first convolution layer.
Display at least eight feature maps.

Task 5
Compare
- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6
Construct the following CNN.

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$$

Train for
- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Architecture used: Conv2D(32,3×3,ReLU) → MaxPool(2×2) → Conv2D(64,3×3,ReLU) → MaxPool(2×2) → Flatten → Dense(128,ReLU) → Dense(10,Softmax). Total trainable parameters = 545,098.

Task 7
Evaluate
- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

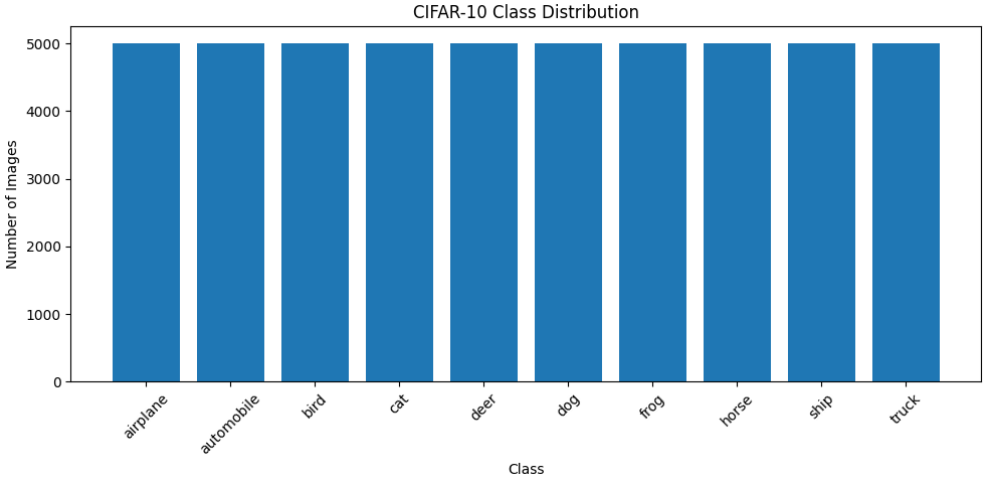
8.1 Source Code

The complete source code for this experiment has been uploaded to a GitHub repository for reference. The repository is periodically updated with code refinements, additional implementations, and detailed insights gained through successive iterations of the experiment.

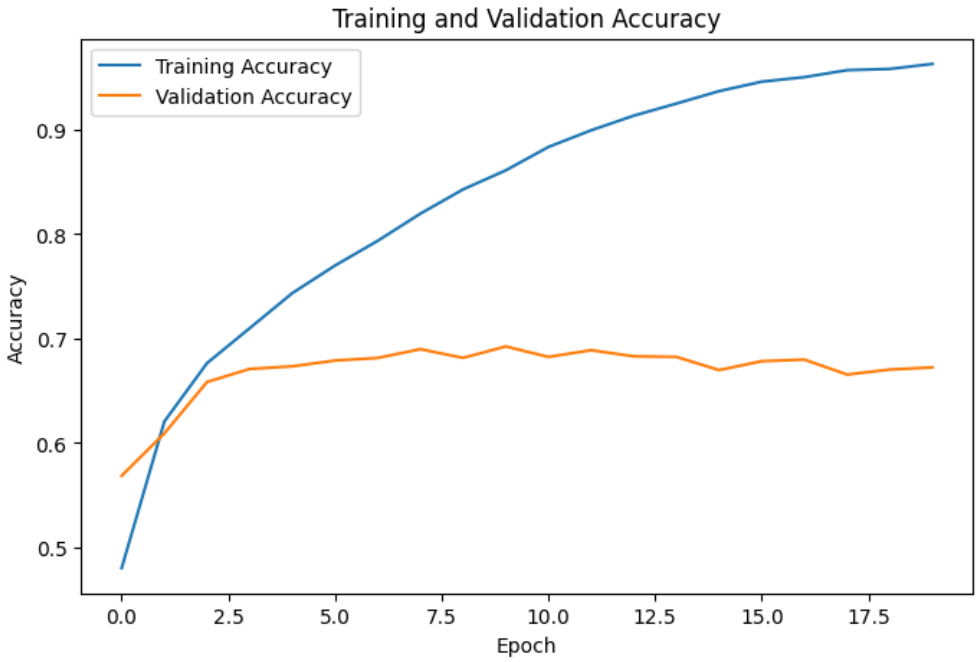
GitHub Repository:
`https://github.com/shravW/Deep-Learning-Lab-CS3807-SEM-5/tree/main/Ex_3_DL`

9. Mandatory Plots





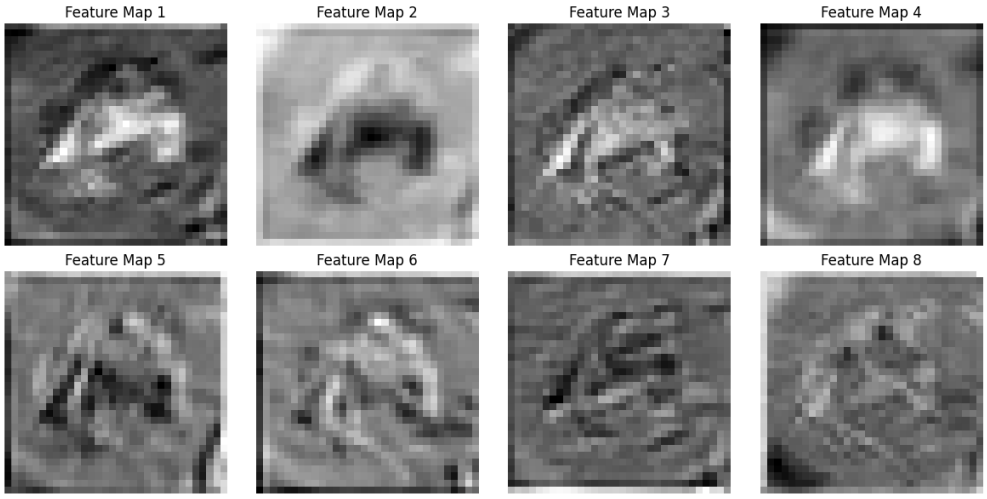
*T*he data appears to be distributed roughly equally across all ten classes, and therefore there is no need to adjust for class imbalance ahead of training.



*T*raining accuracy keeps increasing, while validation accuracy improves initially and then mostly levels off. This suggests that the model continues learning the training data but starts to overfit, with little further improvement on unseen data.



*T*raining loss continuously decreases, while validation loss decreases initially and then starts increasing. This indicates that the model is overfitting as training continues.



*T*he feature maps show that the CNN is detecting different visual patterns and important features from the input image. Different maps highlight areas such as edges, shapes, and textures, showing how the network extracts useful information at different levels.

10. Additional Exercises

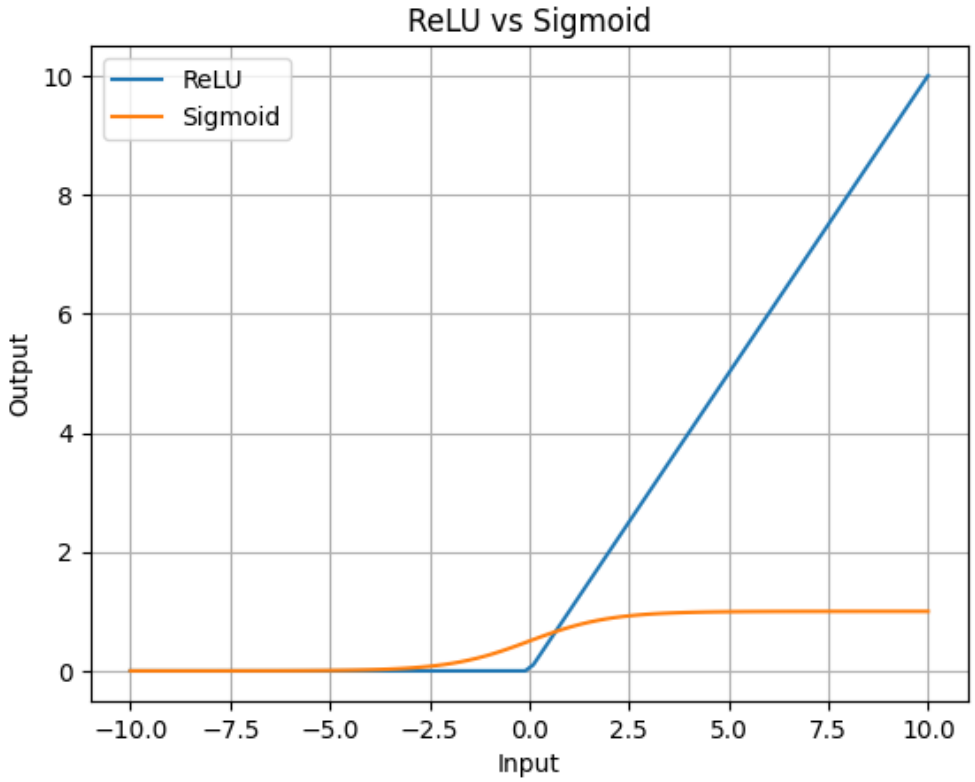
1. **C**alculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

$$\frac{N - F + 2P}{S} + 1 = \frac{64 - 5 + 2 \times 2}{2} + 1 = \frac{63}{2} + 1 = 31.5 + 1 \;\Rightarrow\; 32$$

   Output size $= 32 \times 32$ (using floor division, consistent with Keras' output-shape arithmetic).
2. **C**ompute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

$$(F \times F \times C_{in} + 1) \times \text{Filters} = (3 \times 3 \times 3 + 1) \times 64 = 28 \times 64 = 1{,}792$$

3. **C**ompare ReLU and Sigmoid activation functions. ReLU(x)= $\max(0, x)$ is computationally cheap, does not saturate for positive inputs, and largely avoids the vanishing-gradient problem, which is why it is the default activation in modern CNN hidden layers. Sigmoid squashes outputs to $(0, 1)$ but saturates for large $|x|$, causing very small gradients that slow down training in deep networks; it is mainly reserved for output layers of binary classifiers rather than hidden convolutional layers.



4. **R**eplace Max Pooling with Average Pooling and compare the results. Max pooling performs slightly better than average pooling on this dataset. This suggests that selecting the strongest features is more useful for the model than averaging the features, although the difference in performance is relatively small.
5. **I**ncrease the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time. Increasing the number of convolution filters from 16 to 64 increases the layer's trainable parameters roughly four-fold, typically improving accuracy up to a point of diminishing returns, at the cost of proportionally higher computation time per forward/backward pass and a greater risk of overfitting if the training set is not large enough to support the extra capacity.

11. Results

Accuracy	0.6674
Precision	0.6707
Recall	0.6674
F1-score	0.6675

Classification Report:

	precision	recall	f1-score	support
airplane	0.75	0.67	0.70	1000
automobile	0.78	0.80	0.79	1000
bird	0.59	0.53	0.56	1000
cat	0.45	0.45	0.45	1000
deer	0.62	0.64	0.63	1000
dog	0.54	0.58	0.56	1000
frog	0.67	0.78	0.72	1000
horse	0.68	0.77	0.72	1000
ship	0.84	0.75	0.79	1000
truck	0.79	0.71	0.75	1000
accuracy	-	-	0.67	10000
macro avg	0.67	0.67	0.67	10000
weighted avg	0.67	0.67	0.67	10000

12. Discussion

Answer the following.

1. **Why is convolution preferred over fully connected layers for images?** CNNs are well suited for images because they take advantage of their spatial structure. Convolutional layers use fewer parameters by applying the same filters across different image regions instead of connecting every pixel to every neuron. They also provide sparse connections, parameter sharing, and translation invariance, allowing the network to efficiently learn visual features.
2. **How does stride affect the feature map size?** Using a larger stride can reduce the spatial size more quickly when the filter is larger than 1. However, a stride of 1 is generally better when detailed image structure needs to be preserved and captured at the finest scale.
3. **What is the role of padding?** Same padding keeps the output's spatial dimensions equal to the input and ensures that border pixels are used properly during convolution. In contrast, valid padding adds no padding, so the spatial dimensions decrease after each convolution.
4. **Why is pooling used?** Pooling reduces the size of feature maps, which helps reduce computation and the number of parameters. It also provides some translation and distortion invariance by combining information from nearby pixels.
5. **How do feature maps represent image characteristics?** Feature maps are the outputs produced when filters are applied to an image. Early layers mainly detect simple features like edges, colors, and textures, while deeper layers combine these features to identify more complex patterns.
6. **Why do CNNs require fewer parameters than MLPs?** Dense layers use many more parameters because every input is connected to every neuron. Convolutional layers use small filters and shared weights, which greatly reduces the number of parameters and makes them more efficient for image processing.

13. References

1. Goodfellow et al., *Deep Learning*.

2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.